Design layer

the whole layer, as one document

Contents

1 Fabric PostgreSQL 4

1.1 Foundation 4
1.2 Pending updates 5
1.3 System at a glance 6
1.4 Stored model and authority 7
1.4.1 Persistence aggregate 8
1.4.2 Values and translated projections 9
1.4.3 Row shape 10
1.5 Construction and pool lifetime 10
1.6 Conditional storage and effect timing 11
1.6.1 Read and insert 12
1.6.2 Replacement and lease predicates 12
1.6.3 Isolation, acknowledgement and bounds 13
1.7 Lease renewal and backend time 14
1.8 Projection maintenance 15
1.9 Ready discovery and recovery 16
1.9.1 Parent recovery obligations 17
1.10 Family closure and pruning 18
1.10.1 Membership and retained obligations 18
1.10.2 Concurrent deletion and delayed writes 18
1.11 Diagnostic snapshots 19
1.12 Schema and record compatibility 20
1.12.1 Forward schema installation 20
1.12.2 Reader and writer contracts 21
1.13 Verification and operating limits 22
1.14 End-to-end walkthrough 24

Glossary 24

ROOT DESIGN

Fabric PostgreSQL

1 Fabric PostgreSQL

1.1 Foundation

A shared database preserves Fabric records and coordinates execution owners.

GOAL

Store Fabric work across process and node loss

Provide an Erlang PostgreSQL backend for the Fabric leased-store contract. Multiple store processes share records without both advancing one expected revision.

GOAL

Keep maintenance safe across mixed run families

Discover retained waits, expose useful diagnostics and remove complete settled families while preserving uncertain effects and unresolved ownership.

NO-GOAL

Own the connection pool or business runtime

The application supervises the supplied pool. Fabric owns controllers, effect fences, codecs, approvals, family grants and registered recovery.

NO-GOAL

Provide an application transaction or remote rollback

Record commits use their own database operations. The adapter does not combine a tool effect with its Fabric result, deliver a queue outbox, compensate an external operation or establish exactly-once business effects.

INVARIANT

One expected revision advances at most once

Insert creates revision 1 only when the key is absent. A conditional commit replaces the bytes and increments the revision only when its expected revision and lease condition hold together.

mechanism

INVARIANT

Unconfirmed writes remain distinguishable

Reads return exact record text. An unavailable write is not called definitely refused; Fabric confirms its own target revision and fresh write token through readback before releasing dependent effects.

mechanism

INVARIANT

Metadata never substitutes for record authority

Projections attach to the source revision and current projection version. Unknown or stale metadata does not make a wait eligible or a family prunable.

mechanism

INVARIANT

Pruning removes only a complete settled closure

Every member has current readable retention metadata, reciprocal membership, sufficient record age and no live lease within the pruning transaction. A child is never deleted alone.

mechanism

INVARIANT

Node identity is unique among live VMs

The application assigns a stable node identity that no other live VM using the database shares. Reusing one live identity can let a restarted store mistake another process for its own prior owner.

convention

PRINCIPLE

Keep storage facts separate from execution decisions

The backend owns atomic storage, lease time and queryable projections. A claimed run still requires compatible deployed code and Fabric's policy, ancestry, quota and effect checks.

PRINCIPLE

Retain uncertainty instead of guessing a result

Unreadable records remain unknown. Lost commit acknowledgements remain operational failures until authoritative evidence confirms their outcome. ADR-0001 ADR-0002

1.2 Pending updates

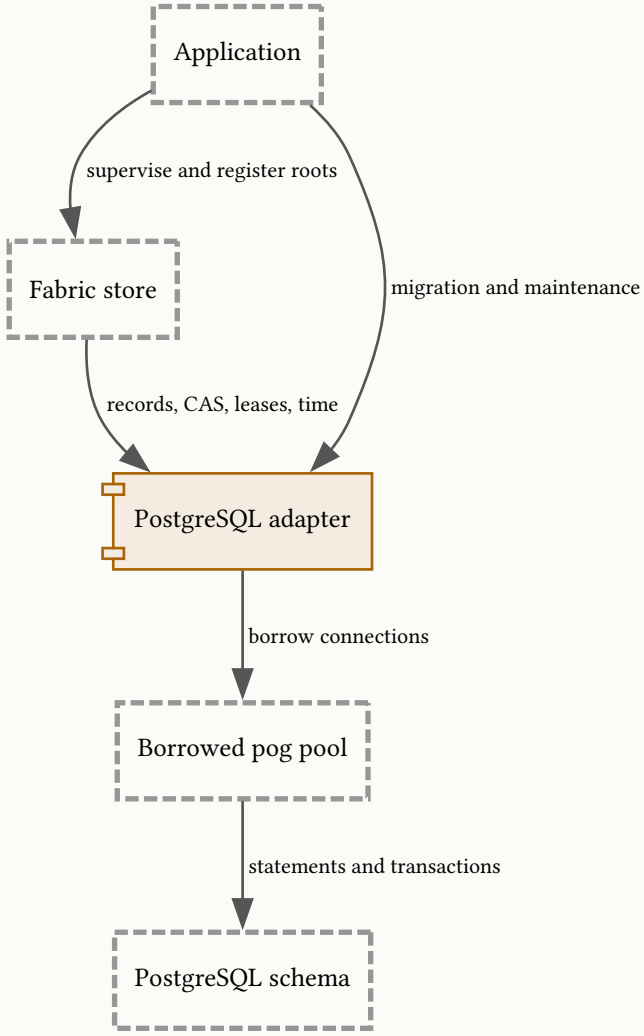
verify **Row shape and maintenance defaults need a code check** 2026-10-17

The row-shape block, the maintenance and retention defaults and the diagnostics rules restate the retired adapter usage guide. Check each column, default and rule against the migrations, the settings module and the adapter tests.

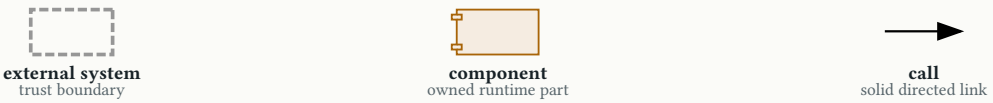
1.3 System at a glance

One persistence context implements the parent store port.

VIEWPOINT RUNTIME · ALTITUDE L1 · BOUNDARY Application-owned pool and Fabric-owned execution



LEGEND



Amber identifies this adapter. Fabric and the application retain separate authority; PostgreSQL is storage infrastructure rather than another controller.

- The adapter is a separate package depending on Fabric and pog/pgo. It imports no Grind runtime and adds no second pool.
- The parent Fabric model owns run identity, graph/agent control, attachments, approvals and family budgets. This context owns their persistence interpretation and database operations.
- The relationship is conformist from this adapter to the Fabric backend and projection contracts. The application is the customer of maintenance APIs and the owner of pool lifetime and global admission.

Fabric leased-backend integration

contract

Retain exact records and return typed storage ownership evidence.

responsibility

Implement the Fabric LeasedBackend without interpreting business callbacks.

interface

now, get, insert, compare_and_set, renew, claim_expired and claim_ready. Keys, bytes, owners, integer TTLs and expected revisions come from the parent port.

interactions

Fabric serializes its work per run and confirms unavailable writes. Core record decoders produce retention, discovery and diagnostic projections.

invariants

Only storage revision authorizes replacement; the lease predicate is an additional condition. Correlation remains retained record data rather than an adapter idempotency key.

failure

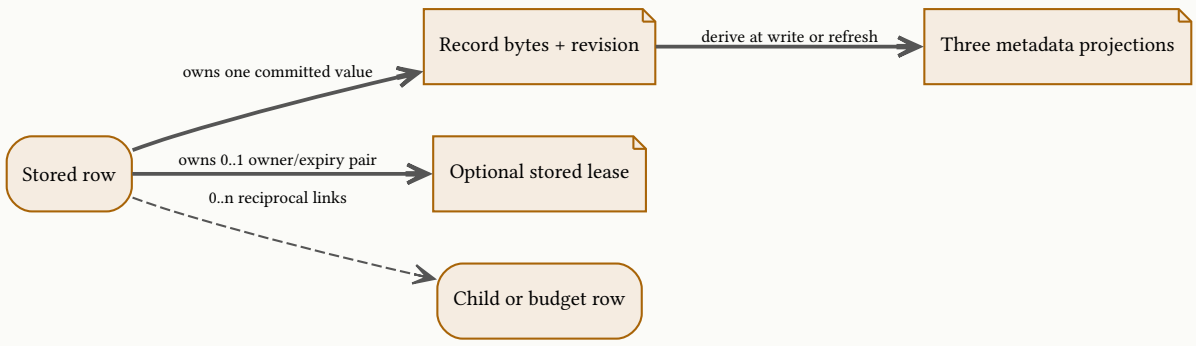
NotFound, AlreadyExists, Conflict(current), LeaseRefused(holder) or Unavailable(reason). The adapter never collapses unknown commit outcome to definite nonexecution.

Unit	Responsibility	Children
Construction	Borrow pool and admit adapter settings	Schema validation, store configuration, supervision order
Conditional storage	Preserve record authority	Read, insert, CAS, lease renewal and expired claims
Metadata maintenance	Derive safe indexed interpretations	Retention, discovery and diagnostic refresh
Ready discovery	Claim eligible free waits	Dependency map, interval and absolute deadline
Family pruning	Delete one complete closure atomically	Membership, settlement, age, lease and concurrent insert checks
Operations	Expose time, diagnostics and schema compatibility	Snapshot, migration, format windows and verification

1.4 Stored model and authority

Record replacement, lease validity and projection freshness are independent axes.

VIEWPOINT DOMAIN-MODEL · ALTITUDE L2 · CONTEXTS One row and its derived interpretations



LEGEND



Arrows describe persistence relationships. Attachment meaning comes from the parent Fabric model; projections are derived views, not copied controllers.

- A Stored row holds Record bytes at one Storage revision. Its Stored lease remains independent of the accepted execution result.

1.4.1 Persistence aggregate

Stored row aggregate · stateful · persistence · owned by PostgreSQL conditional statement

The adapter's aggregate for one encoded execution or budget record.

ATTRIBUTES

authored **Storage key** · Fabric RunId or budget record key
A key is 1–128 ASCII letters, digits, hyphen or underscore. The database rejects malformed direct backend keys; the public Fabric parser owns ordinary input admission.

observed **Committed value** · Record bytes and positive storage revision
Insert starts at 1. Every accepted replacement advances by one, even when supplied record bytes are identical.

derived **Presence** · Stored row presence stored-row-presence · stored-row-lifecycle
A key has no stored evidence before insertion or after complete-family pruning; reads distinguish presence at lookup time.

derived **Lease validity** · Stored lease state stored-lease-state · stored-lease-lifecycle
Validity derives at read from the optional owner/expiry pair and backend time. Lease-only changes do not advance execution revision.

derived **Record timestamp** · Backend wall time
Insert and accepted record replacement set it at write. Maintenance and ownership-only operations preserve it.

derived **Metadata** · Retention, discovery and statistics projections
Each view derives at write or explicit refresh from actual supported decoders and retains its source revision.

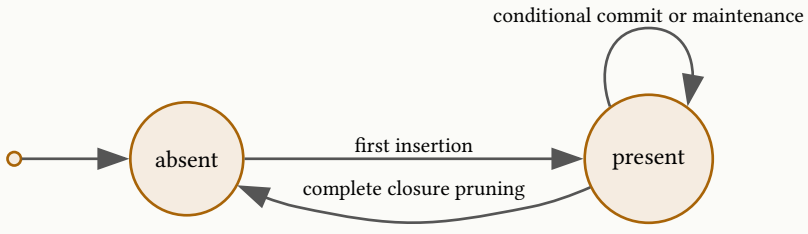
RELATIONSHIPS

`1 : 0..1` A retention projection identifies an immediate parent through its saved reciprocal attachment.

`1 : 0..n` A row can declare children or its matching budget ledger; every retained child has one checked immediate parent.

Stored row presence absent · present

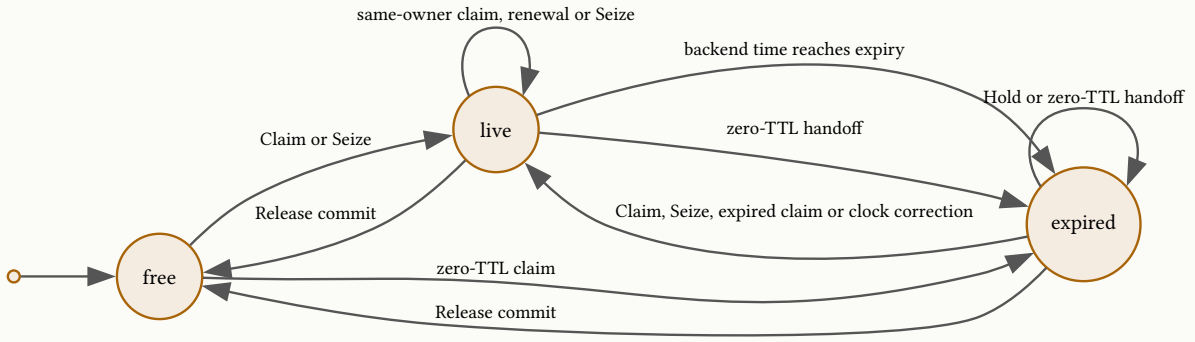
STATE MACHINE Record presence before insert and after pruning



An absent key returns NotFound. Pruning leaves no permanent historical-id reservation; application id-reuse discipline survives beyond the stored evidence window.

Stored lease state free · live · expired

STATE MACHINE Ownership expiry, transfer and release



Expiry is a time-derived observation, not a record write. Hold accepts the same owner even after expiry. Renewal accepts only a live lease. Clock correction can restore apparent live status before another owner replaces the pair.

- Presence is vacant before insert and after pruning; get returns NotFound in both cases. The adapter keeps no permanent tombstone or reserved-key registry after deletion.
- The lease columns are both null or both present. The record revision, owner identity and live flag answer separate questions; a live lease never proves a business result.
- The phase tag is derived by Gleam JSON parsing and may be null. It supports queries but does not decide settlement or family membership.

1.4.2 Values and translated projections

Metadata projection value-object immutable persistence owned by Fabric projection decoder; adapter attaches source revision

A validated storage view of parent-owned record semantics.

ATTRIBUTES

derived **Interpretation identity** · Projection kind, version and source revision
Retention, discovery and statistics have independent versions. Freshness requires both the current kind version and equality to the row revision.

derived **Validated interpretation** · Known metadata or unknown marker
Unsupported, corrupt or misfiled records retain the version alone. Retention refresh also maps a missing-parent orphan to unknown.

RELATIONSHIPS

n : 1 Each projection belongs to one exact source row revision.

Wait observation value-object immutable persistence owned by ready claim

Scheduling evidence retained alongside one discoverable wait.

ATTRIBUTES

observed **Observation scope** · Opaque key, dependency revision map and claim time
The claim records these values with its lease. Dependency entries include null when a referenced child is absent.

Refinement or sum	Values	Meaning
Current	Revision, exact bytes, Holder	One authoritative read; Holder is Free or Held(owner, live).
Lease request	Hold / Claim / Seize / Release	Each command selects a distinct conditional-write ownership contract.

Refinement or sum	Values	Meaning
Projection validity	Stale / known / unknown	An old source revision or decoder version is stale; current undecodable data is unknown.
Discovery trigger	Changed(dependencies, deadline) / Poll(interval, deadline) / At(due)	Optional absolute deadline makes changed or polling waits eligible independently.
Record age	Nonnegative displayed milliseconds	Diagnostics clamp negative wall-clock differences to zero. Pruning uses its own timestamp comparison.
Migration state	Greatest applied version	A forward schema ledger independent of execution and projection versions.

- The adapter emits no domain events and stores no deployment callbacks. Fabric originates lifecycle observations after its own confirmed transitions.
- Settings are an immutable value object; diagnostics are an immutable observation. A pruning closure is a transaction-local aggregate view over parent-owned family membership, not a persisted second family entity.

1.4.3 Row shape

The runtime migrations and the retained cigogne migrations are the executable schema definitions, and tests compare their statements exactly. The block below states the current row shape.

CODE SQL

```sql
CREATE TABLE fabric_runs (
  run_id text PRIMARY KEY CHECK (run_id ~ '^[A-Za-z0-9_-]{1,128}$'),
  revision bigint NOT NULL CHECK (revision >= 1),
  record text NOT NULL,          -- the exact bytes written
  phase text,                    -- the record's phase tag, for queries
  retention jsonb,               -- Fabric's validated family projection
  retention_revision bigint,     -- source revision of that projection
  parent_id text GENERATED ALWAYS AS (retention ->> 'parent') STORED
    REFERENCES fabric_runs(run_id) DEFERRABLE INITIALLY IMMEDIATE,
  discovery jsonb, discovery_revision bigint,
  statistics jsonb, statistics_revision bigint,
  dependency_ids jsonb GENERATED ALWAYS AS (discovery #> '{wait,dependencies}') STORED,
  observed_key text, observed_dependencies jsonb,
  discovery_checked_at timestamptz NOT NULL DEFAULT '-infinity',
  lease_owner text, lease_until timestamptz,
  updated_at timestamptz NOT NULL DEFAULT clock_timestamp(),
  CHECK ((lease_owner IS NULL) = (lease_until IS NULL)));
-- indexes: expired leases, settled roots by age, immediate parents, idle dependencies
```

1.5 Construction and pool lifetime

Pure settings borrow one application resource and defer startup to supervision.

- A PostgreSQL settings value describes the borrowed resource; constructing it does not establish database readiness.

Settings and store construction

responsibility

Describe a connection, stable VM identity, lease duration and schema without allocating resources.

interface

settings(connection, node:), with_lease(Duration), with_schema(String), store(Name, Settings) and backend(Settings). Defaults are schema public and lease 30 seconds.

interactions

store delegates validation and store-value construction to Fabric store.leased. backend exposes the same port for wrappers and conformance; it does not start the pool or apply migrations.

invariants

Schema is 1–63 lowercase letters, digits or underscore, with no initial digit. Quoted schema identifiers follow validation; record keys and values remain query parameters.

failure

with_schema returns InvalidSchema. store returns Fabric LeaseConfigError for invalid node or lease. Database reachability and migration remain operational checks.

Setting	Admitted value	Owner
Node	1–128 ASCII letters/digits or `._-@:`; excludes `nonode@nohost`	Fabric store validates; application assigns stable unique identity.
Lease	100 ms to 2^32 – 1 ms	Fabric store validates Duration and renews every third of it.
Schema	Lowercase plain identifier up to 63 characters	Adapter validates before SQL construction.
Connection	pog.Connection supplied by application	Application owns configuration, pool size, startup, availability and shutdown.

- Use a named connection under an application supervisor. The pool precedes the store subtree in rest-for-one order, so a pool restart also restarts the store.
- With automatic recovery, sweeper.supervised replaces the bare store subtree and supervises store before sweeper. Shutdown stops sweeping before runners drain and keeps the pool available until handoffs finish.
- Apply migrations and any required projection refresh before enabling recovery and request admission. Store construction is not schema readiness; store.readiness supplies the parent's bounded reachability and acceptance evidence.
- The adapter has no service-wide concurrency limit, connection-checkout policy or whole-operation deadline. Fabric's backend timeout bounds its worker wait; a caller supplies pool and database bounds for direct maintenance calls.

APPLICATION DELIVERY RETAINS ITS OWN TRANSACTION BOUNDARY

An existing transactional job or outbox bridges accepted application work to a keyed Fabric invocation. The supplied connection does not combine application acceptance with a Fabric start. Registered parent recovery owns admitted runs; an idempotent application transaction incorporates the retained outcome before pruning. A durable application applied marker prevents reinvoking an applied turn after its Fabric record is removed. The parent invocation contract also permits an application graph operation for explicit result incorporation.

BEHAVIOR

Invalid schema creates no storage effect

When A caller selects a schema identifier outside the admitted form.

Then with_schema returns InvalidSchema.

Then No database operation occurs.

level: interface area: Construction

BEHAVIOR

Invalid lease settings create no store value

When A caller constructs a store with an invalid node identity or lease duration.

Then The parent's typed LeaseConfigError is returned.

Then No store process or pool is started.

level: interface area: Construction

ADR-0001

1.6 Conditional storage and effect timing

Each normal write uses one autocommitted statement with one expected revision.

- A Conditional commit retains authority at one expected revision; the Lease owner adds the requested execution-ownership condition.

1.6.1 Read and insert

Exact-record read and first insertion

responsibility

Return the current committed value or install the first value at a key.

interface

get returns Current or NotFound; insert writes revision 1 or returns AlreadyExists. Insert Claim/Seize installs owner and expiry; Hold/Release inserts free.

interactions

The write derives phase and all three projections in Gleam before storing them with the bytes and revision. The parent owns record encoding and the fresh write token.

invariants

Text is neither normalized nor parsed as a PostgreSQL execution JSON object. Whitespace, escaping and Unicode return unchanged; escaped NUL stays valid text.

failure

Malformed direct backend keys and storage/decoding failures return Unavailable. Unknown record formats may be stored, but metadata remains unknown and phase may be null.

- INSERT uses ON CONFLICT DO NOTHING. One changed row confirms insertion; an existing key is AlreadyExists. A serialization failure reads back existence and retries only an absent key within the finite retry allowance.
- get derives Held(owner, live) from lease_until greater than clock_timestamp(). It does not claim, renew, migrate, refresh or recover anything.

1.6.2 Replacement and lease predicates

Expected-revision replacement

responsibility

Atomically replace one row and apply its requested lease condition.

interface

compare_and_set(key, expected, bytes, Lease) stores expected+1. The expected revision is checked before lease refusal is classified.

interactions

One statement replaces bytes, phase, record timestamp and three projections with their new source revisions. Lease mutation, when requested, is in that same statement.

invariants

Two writers using one expected revision cannot both succeed, including across pools and machines. A lease-only operation never increments this revision.

failure

No changed row is read back as NotFound, Conflict(current) or LeaseRefused(holder). If the latest read permits the condition, retry is bounded; exhausted still-permitted retries are Unavailable.

Lease command	CAS predicate after expected revision	Committed lease
Hold(owner)	Current owner equals supplied owner; expiry may be past	Unchanged
Claim(owner, ttl)	Free, same owner, or expired	Supplied owner; expiry is backend now plus TTL
Seize(owner, ttl)	No additional owner condition	Supplied owner; expiry is backend now plus TTL
Release	No additional owner condition	Both owner and expiry cleared

- Seize allows Fabric's explicit cancellation and same-node restarted-store protocol. It is an advanced parent-port command rather than a new adapter authentication capability.
- A handoff that needs automatic expired recovery uses Claim(owner, 0), retaining a held-but-expired lease. Release instead leaves a free wait for ready discovery or explicit command.
- The body-start fence is a Fabric record transition stored through CAS. The adapter neither calls the body nor proves a downstream service honors an idempotency key.

1.6.3 Isolation, acknowledgement and bounds

- Normal statements are written for READ COMMITTED and use the borrowed connection's default isolation. A concurrency failure with SQLSTATE 40001 or 40P01 changed nothing; insert/CAS read back, and renew/claims retry.
- The normal statement allowance is the initial attempt plus up to five retries. Another database or connection failure becomes Unavailable immediately rather than an unconditional replay.
- Unavailable includes timeout and missing connection. A late statement or lost commit reply may still leave a committed write; Fabric's exact-byte, target-revision readback owns confirmation.
- The adapter never retries an arbitrary write solely because it is unavailable. Its retry predicates preserve the original key, expected revision and encoded bytes. Core runner retries and backend-call bounds remain separately owned by Fabric.

Effect	Commit boundary	Preserved state
Insert / CAS	One autocommitted conditional statement	Record authority changes together with phase and projections.
Renew / expired claim	One autocommitted owner/expiry update	Bytes, revision, record timestamp and projection sources stay fixed.
Ready claim	One autocommitted lease and observation update	Bytes, revision and retention age stay fixed.
Refresh	One locked transaction per batch	Bytes, execution revision, lease and record timestamp stay fixed.
Prune	Serializable transaction	Whole selected closures delete together, or the transaction aborts.
Migrate	One advisory-locked transaction	All unapplied forward steps commit together.
Time / stats	Read only	No execution or ownership mutation.

BEHAVIOR

Competing revisions have one winner

Given Two callers hold the same current expected revision.

When They attempt concurrent conditional commits.

Then At most one advances that revision.

Then A loser receives conflict or operational uncertainty without releasing a successor effect.

level: boundary area: Record commits

BEHAVIOR

A refused lease condition changes no record

Given The expected revision is current but the declared lease condition is false.

When A conditional commit is attempted.

Then LeaseRefused includes the observed holder.

Then The record and revision remain unchanged.

level: boundary area: Record commits

BEHAVIOR

Record readback preserves confirmation evidence

Given A record has committed.

When The same key is read.

Then The exact encoded bytes and current revision are returned.

Then Metadata interpretation does not rewrite the record.

level: boundary area: Record reads

ADR-0002

1.7 Lease renewal and backend time

Database wall time judges expiry while Fabric owns local self-fencing.

- Backend time supplies every lease and automatic wait comparison.

Ownership maintenance

responsibility

Extend eligible leases and transfer expired work without changing execution state.

interface

renew(owner, keys, ttl) returns renewed keys; claim_expired(owner, ttl, limit) returns claimed keys. Empty renew input and nonpositive claim limit return an empty list.

interactions

Renewal checks exact owner and a live expiry. Expired claiming orders held expired rows by lease_until and locks at most limit candidates with SKIP LOCKED.

invariants

Concurrent claims are disjoint while their acquired leases remain live. A delayed renewal cannot revive a lease already handed off as expired. Ownership maintenance does not reset record age.

failure

Serialization/deadlock retries are finite; other query failures are Unavailable. A claimed identity must still be read and validated by registered recovery.

Backend clock

responsibility

Supply the same UTC time domain as lease and ready-claim eligibility.

interface

LeasedBackend.now, exposed through Fabric store.now, returns integer Unix milliseconds.

interactions

SELECT floors the epoch value of clock_timestamp(). Leases use that database clock for expiry and renewal; deadlines use it for arming and acceptance.

invariants

No database mutation or migration is needed for this clock read. Failure never substitutes local caller time. Wall-clock samples are not monotonic.

failure

Unavailable covers query failure or a result other than one timestamp. Clock correction may change when a stored absolute deadline appears due.

- The parent store renews runner leases every lease/3 and self-fences using its monotonic safety window. Lost renewal kills the local runner and its linked model/tool tasks; the adapter only reports renewal evidence.

- Live foreign leases make active work look Working to another store. A prior local process's lease can be seized by the parent's same-node/name protocol; otherwise takeover waits for expiry.
- Families have independent member leases. A live parent does not prevent an expired child from recovery, and a child's takeover does not replace the live parent's application context.

BEHAVIOR

Expired leases cannot be renewed

Given A lease is expired or belongs to another owner.

When The former owner attempts renewal.

Then The key is absent from the renewed list.

Then The lease and execution revision are unchanged.

level: boundary area: Lease maintenance

BEHAVIOR

Clock failure does not invent time

When The backend cannot return its timestamp.

Then The caller receives Unavailable.

Then No local-clock fallback or execution mutation occurs.

level: interface area: Backend time

ADR-0002

1.8 Projection maintenance

Validated decoder output is rebuildable while execution evidence remains untouched.

- A Metadata projection remains attached to its inspected revision. An Unknown projection retains the current decoder version without supplying a usable interpretation.

Versioned metadata refresh

responsibility

Attach current retention, discovery or statistics metadata to stale rows in bounded batches.

interface

refresh_retention, refresh_discovery and refresh_statistics return examined row count or RefreshError. Limits must be positive.

interactions

Select stale source revisions or versions in run-id order under row locks with SKIP LOCKED. Decode exact current records and condition updates on the inspected revision within the batch transaction.

invariants

Bytes, revision, lease owner/expiry and updated_at remain fixed. Unreadable records are examined once per decoder version and remain unknown. Normal writes atomically maintain all three views.

failure

RefreshLimitNotPositive rejects before database work; transaction failure becomes RefreshFailed. A missing-parent orphan becomes current unknown retention metadata rather than aborting its batch.

Projection	Current version	Validated data used by adapter
Retention	13	Parent, opaque attachment, every child link and definite settlement. Includes matching root budget declarations.

Projection	Current version	Validated data used by adapter
Discovery	14	Opaque wait key plus dependencies, poll interval and/or absolute due time. Includes graph and idle-agent approval expiry.
Statistics	3	Run or budget kind; active/waiting/finished; approval/reconciliation flags. No prompts or outputs.

- Freshness is checked separately for each view; schema version is not its decoder version. A legacy backend write increments execution revision while leaving old metadata stale.
- Refresh is transactional but its count measures examined rows, including current unknown markers. Concurrent refreshers skip locked rows, so a zero count describes this call rather than proving no other batch remains in flight.
- Retention refresh uses serializable isolation with at most five transaction attempts. Discovery and statistics refresh use a normal transaction and expose its failure; they have no separate automatic serialization retry loop.
- Discovery refresh clears dependency observations to reestablish a baseline. For the same saved polling key it preserves observed_key and discovery_checked_at, so refresh cannot make an already observed poll immediately due.
- Attachment keys remain escaped JSON text in metadata. A provider call id containing NUL never becomes a PostgreSQL NUL character in the index, while original record text stays unchanged.

BEHAVIOR

Refreshing metadata preserves execution evidence

Given A stored record needs a metadata-version or source-revision refresh.

When A bounded refresh examines it.

Then Its original bytes, revision, lease and record age remain unchanged.

Then An unreadable interpretation remains explicitly unknown.

level: boundary area: Metadata refresh

BEHAVIOR

Invalid maintenance limit fails before database work

When A caller requests a nonpositive refresh limit.

Then RefreshLimitNotPositive is returned.

Then No rows are examined or changed.

level: interface area: Metadata refresh

ADR-0003

1.9 Ready discovery and recovery

Claims observe triggers atomically and leave execution decisions to Fabric.

- A Ready claim saves a Wait observation so later dependency changes remain visible.

Ready-claim selection

responsibility

Acquire free, current discoverable waits whose retained triggers are eligible.

interface

claim_ready(owner, ttl, limit) returns at most limit keys, or an empty list for a nonpositive limit.

interactions

One candidate CTE reads dependency revisions, locks candidate rows and saves observation key, dependency map, claim time and lease together. Order is least recently checked, then run id.

invariants

Candidates require lease_owner null, current discovery source revision and decoder version. Claims change no record bytes, execution revisions or retention ages. Concurrent claimers skip locked candidates.

failure

Unavailable leaves the claim outcome operationally uncertain. Unknown/stale records cannot become candidates. A claim is scheduling evidence and never permission to recreate an acknowledged missing child.

Trigger	Eligible condition	Retained observation
Changed	Observation key is unseen/different or dependency revision map differs	Each unfinished child id maps to its current revision or null if absent.
Poll	Unseen/different key, or claim time plus interval is at or before backend time	Same-key writes and refresh preserve last claim time.
At	Backend Unix milliseconds reaches the absolute due time	Claim does not consume or move the deadline.
Changed/Poll plus deadline	Either ordinary trigger or absolute time is eligible	Cleanup may retain a distinct dependency/poll key with no deadline.

- An observation key excludes execution incarnation and record revision, so recovery of an unchanged wait does not manufacture new work. New activations or observation scopes have different keys.
- A concurrent dependency update after the observed map remains different on the next scan. Creation, change or disappearance of a child can all trigger observation through its revision/null entry.
- Only held expired rows enter claim_expired; only free waits enter claim_ready. Reads alone neither expire a wait nor claim it.
- Absolute eligibility remains after lease release until the execution leaves the wait. Recovery samples time again; a backward correction can make the wait early, release it and retain its original due time.
- Intervals are lower bounds on eligibility after the previous ready claim. Load, scan interval, callback rebuilding and failed recovery can make actual observation later. No fairness or exact-time scheduling is promised.

1.9.1 Parent recovery obligations

- Register one root factory per runtime kind/definition. Recovery follows retained reciprocal attachments to the root; graph factories rebuild all managed runtimes against the pinned store. Context and credentials are never stored.
- The parent sweeper runs at boot and serially after each interval, claiming at most 50 expired leases and 50 ready waits. It bounds each root recovery to 30 seconds including at most 5 seconds rebuilding context.
- Unknown definitions or failed recovery leave claimed leases to expire for another attempt. A live foreign parent keeps its context while independently recoverable children are inspected.
- Waiting signals with no deadline require explicit delivery. Manual jobs with no deadline require explicit observation. Owned-job cleanup uses its retained cancellation/poll protocol; a stop request alone does not prove remote settlement.
- Graph child, fork, job and signal deadlines retain cleanup independently from expiration. Approval deadlines apply to idle graph or agent questions; unbuilt whole-agent time and family-token limits remain parent design gaps.

BEHAVIOR

Stale discovery cannot grant a ready claim

Given A wait's source revision or discovery version is stale.

When Automatic ready discovery scans it.

Then It is not claimed through that stale interpretation.

Then An explicit refresh or current record write is needed to restore eligibility.

level: boundary area: Ready discovery

BEHAVIOR

Claiming a deadline does not consume it

Given A free wait has reached its absolute deadline.

When It is claimed and later released without leaving that wait.

Then Its original deadline remains retained.

Then It remains eligible when backend time reaches that deadline.

level: boundary area: Ready discovery

ADR-0003

1.10 Family closure and pruning

Finished status alone is insufficient authority to delete retained evidence.

- A Pruning closure includes every retained member before deletion. Each member's Record age must satisfy the selected retention interval.

Pruning transaction

responsibility

Delete up to a requested number of complete settled root closures without orphaning children or discarding uncertainty.

interface

prune(settings, ended_for: Duration, limit: Int) returns deleted row count. Age must be nonnegative and limit positive; the limit counts root families, while the result counts all deleted rows.

interactions

Serializable recursive selection follows current retention parent links. Eligible roots order by updated_at then id, lock with SKIP LOCKED and delete their complete member set in one transaction.

invariants

Every member is current/readable, definitely settled, old enough and not live-leased. Parent and child attachment keys must agree in both directions; every declared child must exist.

failure

PruneAgeNegative and PruneLimitNotPositive reject before the transaction. PruneFailed includes operational failure; lost commit acknowledgement can leave deletion count unknown even if deletion committed.

1.10.1 Membership and retained obligations

- Use saved attachments rather than run-id prefixes. Hashed graph children, managed agents, delegated descendants and admitted fork branches remain members despite unrelated-looking keys.
- The matching budget ledger is a retained child of its declaring root with an attachment encoding the same limits. It is never independently pruned; initialized roots cannot lose the ledger and regain capacity.
- Missing, misfiled, unreadable, unacknowledged or wrongly attached members retain the entire root. A terminal agent with an uncertain action and a graph with unresolved owned cleanup remain unsettled.
- Every new record write restarts that member's retention interval. Projection refresh, lease renewal and ready claims do not make an old execution young.
- One member's recent settlement or live lease prevents deleting the family. Another eligible family may still be selected; a blocked old root does not consume the selection limit.

1.10.2 Concurrent deletion and delayed writes

- Pruning and retention refresh force SERIALIZABLE isolation inside their own transactions. Serialization failure or deadlock retries the whole transaction at most five total attempts.
- Competing pruners skip roots held by another transaction. Concurrent member writes or renewals participate in serializable revalidation rather than being ignored after root selection.
- A generated immediate parent foreign key is DEFERRABLE INITIALLY IMMEDIATE. It prevents a delayed child insertion from creating an orphan after its parent was deleted.

- The database constraint does not authenticate attachment meaning; the pruning query also checks both reciprocal retained keys. A current retention marker alone is insufficient.
- Pruning ends the family's replay and reopening window. Applications must not reuse deleted logical ids for different executions that could receive old messages; the adapter does not enforce a permanent historical-id registry.
- Repeating prune after a lost commit response remains safe, but cannot recover the previous deleted-row count. Non-serialization errors are not treated as definitely rolled back at the commit-reply boundary.

BEHAVIOR

An incomplete family retains every member

Given A closure has missing, stale, unreadable, unresolved, recent or live-leased membership.

When Pruning inspects its root.

Then No member of that closure is deleted.

Then Other complete eligible roots may still be pruned.

level: boundary area: Family pruning

BEHAVIOR

Pruning counts rows within a family limit

Given Several complete closures qualify.

When A caller prunes with a positive root limit.

Then At most that number of root families is selected oldest first.

Then The returned count includes their descendants and budget records.

level: interface area: Family pruning

ADR-0003

1.11 Diagnostic snapshots

A database observation measures records and lease backlog without controlling them.

- A Diagnostic snapshot describes one sampled database state without changing its records or owners.

Database-wide statistics

responsibility

Describe current individual-record execution groups, intervention evidence and ownership backlog.

interface

stats(Settings) returns statistics.Snapshot or StatsFailed. Snapshot contains sampled_at_ms, seven Group values, budget_records, leases_per_node and ExpiredLeases.

interactions

One SELECT uses one MVCC snapshot and one materialized clock sample. It reads the small statistics projection and lease/timestamp columns, not record bodies.

invariants

Read only: no refresh, write, claim, renewal or recovery. Current projection requires matching source revision/version and validated kind/classification flags; other rows count unknown.

failure

Query failure and malformed/non-single result become StatsFailed. Failure never returns a fabricated empty snapshot.

Group	Condition	Interpretation
working	Active record with live lease	Lease evidence; does not prove a body is currently running.

Group	Condition	Interpretation
unattended	Active record with no live lease	Work needs a runner rather than ordinary parked wait.
waiting	Readable wait classification	Signal/job/child/approval waits can be validly idle.
finished	Readable finished classification	Terminal control; may still require reconciliation.
approval	Own pending approval evidence	May overlap waiting or another execution group.
reconciliation	Own unresolved-effect evidence	May overlap finished; child flags are not copied onto ancestors.
unknown	Stale, unsupported, corrupt or misfiled diagnostic interpretation	Explicit missing knowledge, never healthy zero.
budget_records	Current budget kind	Excluded from run groups.

- Group(count, oldest_record_age_ms) counts individual rows including children. An empty group has no age; a nonempty age measures its oldest latest record write, not time inside a phase.
- NodeLeases(node, count) aggregates live lease owners by their node prefix before the slash. Lease gauges include all rows with live ownership, including rows whose diagnostic interpretation is unknown.
- ExpiredLeases(count, oldest_overdue_ms) counts held leases at or before the sample. Empty backlog has no oldest overdue age.
- Displayed ages floor milliseconds and clamp negative values to zero when wall clock moves backward. The one sample keeps every group comparison internally consistent.
- store.readiness is a separate parent-owned local admission/runner/lease-safety read. Neither readiness nor statistics proves shutdown drain completion or resumes a finished uncertain run.

BEHAVIOR

Statistics is an observation without maintenance

When A diagnostic snapshot is read.

Then Records, revisions, leases, ages and scheduling observations remain unchanged.

Then Unknown interpretations remain visible as unknown.

level: boundary area: Diagnostics

BEHAVIOR

Statistics failures cannot appear as empty health

When The database or snapshot decoder cannot supply a diagnostic snapshot.

Then StatsFailed is returned.

Then No zero-count substitute is returned.

level: interface area: Diagnostics

ADR-0004

1.12 Schema and record compatibility

Storage schema, record envelopes and derived-view versions have different upgrade owners.

- A Schema migration changes physical storage. Encoded record compatibility and derived interpretations remain separate contracts.

1.12.1 Forward schema installation

Schema migration

responsibility

Create the configured schema if absent and apply every unapplied numbered step atomically.

interface

migrate(Settings) returns Nil or MigrationFailed. Equivalent cigogne up statements remain in priv/migrations and its configuration.

interactions

The transaction sets READ COMMITTED, takes a schema-specific advisory lock, sets local search_path to only the target schema and reads the greatest recorded version. It applies later built-in steps and records each.

invariants

Concurrent migration callers serialize per schema. A precreated schema needs no database CREATE privilege. A schema migrated further by a newer adapter is left unchanged.

failure

Statement failure rolls back the transaction; failure receiving commit acknowledgement can leave outcome unknown. Repeating migrate is safe. Migration query helpers use 60-second query timeouts; this is not a single 60-second transaction deadline.

- Every step takes the same transaction-scoped advisory lock derived from fabric-postgres-migrate: plus current schema. The runtime's outer lock uses the same identity before schema creation. Hash collisions can serialize unrelated schemas but do not grant another schema's mutation authority.
- Runtime migration is forward only. Cigogne files also contain explicit down statements for application-managed reversal; their presence does not make runtime downgrade or newer record rewriting safe.
- Append a new numbered step and matching SQL file when changing schema. Released statements remain immutable; migration tests compare the runtime and cigogne up statements exactly.

Schema version	Owned storage transition	Compatibility consequence
1	Exact text, revision, phase, paired lease and timestamp; migration ledger	Baseline persistence. Historical prefix root index is no longer current authority.
2	Validated retention, source revision and parent foreign key	Replace prefix membership; refresh existing rows before pruning.
3	Discovery metadata, scalar dependency and observation baseline	Existing rows remain unindexed until refresh.
4	Index scheduled polling waits	Same-key poll timing is retained independently of record writes.
5	Index absolute due waits	Due time is not a polling interval.
6	Replace scalar dependency/observation with dependency array/map	Stop older backend writers before migration; refresh before idle discovery.
7	Statistics projection and source revision	Existing rows remain diagnostic unknown until refresh.

1.12.2 Reader and writer contracts

Format	Read / write window	Meaning retained
Agent	Read 1–9; writer selected 2–9, default 9	v4 provider data, v5 tagged graph parent, v6 settled-child evidence, v7 family declarations and quota outcomes; v8 initial-input boundary; v9 derived child run ids.
Graph	Read 5–15; current writer 15	Separate graph envelope; agent writer selection does not downgrade it.
Retention	Current projection 13	Decoders, family declarations, all admitted fork members, deadlines and definite cleanup.
Discovery	Current projection 14	All unfinished fork dependencies, interval/ deadline triggers and approval expiry.
Statistics	Current projection 3	Run/budget and execution/intervention classification.
Schema	Current 7	Physical columns and indexes required by adapter operations.

- store.with_record_version must configure the returned store value before starting it; all handles and the sweeper use that value. Existing values and runners keep their own writer setting. Unsupported writer requests return UnwritableVersion(requested, oldest, newest).
- Older writers refuse a state whose meaning they cannot retain before changing its record or dispatching its successor. Reader rollout precedes enabling capabilities that need a newer envelope; writer selection never rewrites existing rows.
- Agent v2 can retain a never-started cancelled child in its historical empty cancelled shape. The genuine v2 decoder validates compatible writes and exact-byte readback, rather than assuming current decoding establishes old-reader compatibility.
- Graph v7 adds read-only job waits; v8 scheduled intervals; v9 owned jobs and stop requests; v10 signal deadlines; v11 job deadlines; v12 child deadlines; v13 fork membership; v14 fork deadlines; v15 correlation, family root and approval evidence/deadlines.
- Older optional fields stay absent on read: a historical manual job remains manual, and a historical approval or wait without deadline gains none. Saved codecs, definitions and backend lease protocol must also be compatible; record version alone is insufficient.
- Downgrading after newer writes needs a separate migration plan. Schema migration does not change encoded record bytes; projection refresh does not make a newer record readable by an old runtime.

BEHAVIOR

Concurrent migration applies each step once

Given Several nodes target the same schema.

When They request forward migration concurrently.

Then Each unapplied step is recorded once.

Then Later callers observe the completed schema without repeating it.

level: boundary area: Migrations

BEHAVIOR

A newer schema is not downgraded

Given The schema ledger contains a version beyond this adapter's built-in steps.

When This adapter runs migrate.

Then The newer schema remains unchanged.

Then No reverse migration is attempted.

level: boundary area: Migrations

ADR-0001 ADR-0005

1.13 Verification and operating limits

Database behavior needs real concurrent tests; pure compilation proves fewer properties.

Retained adapter verification

responsibility

Exercise the public adapter and parent backend contract against disposable PostgreSQL.

interface

scripts/test-postgres.sh runs this package's tests inside the parent Nix shell. The compiled example runs as an integration test.

interactions

The harness removes PG variables, creates a private temporary PostgreSQL 16 cluster, supplies only its own test URL and removes the cluster on exit. Peer tests kill a separate Erlang VM to inspect recovery without tool replay.

invariants

No existing database is selected from environment. Each conformance factory creates fresh empty storage; independent checks cannot consume each other's claim candidates. Sources, SQL, fixtures and FFI remain executable evidence.

failure

Direct gleam test fails without the harness URL. Cluster startup failure refuses rather than attaching to another server; failed runs retain relevant server output in the gate log.

Evidence area	Retained scenarios	Limit
Backend	Shared conformance, byte identity, malformed ids, blocked READ COMMITTED/REPEATABLE READ writers and renewal races	Named scenarios establish the tested contracts, not every possible database failure.
Lifecycle	Approval across restart, lost write replies, independent node takeover and peer SIGKILL	Local process/VM crash recovery does not prove remote effect nonexecution.
Family	Missing/mismatched children, NUL attachment keys, budget ledger, concurrent prune and member renewal	Settlement is decoder evidence plus reciprocal membership, not a phase-only predicate.
Waits	Lost wakeups, all fork members, job/signal/child/fork deadlines and nested cleanup across two store losses	Discovery is eventual eligibility; it is not exact wall-clock scheduling.
Operations	Readiness, clock nonmutation, stale projections, statistics errors and concurrent refresh	Counts measure individual records and known evidence, not service availability.
Configuration	Schema isolation, concurrent migration, settings/password inspection and compiled README	No claim of a broader pool or credential-redaction security model.

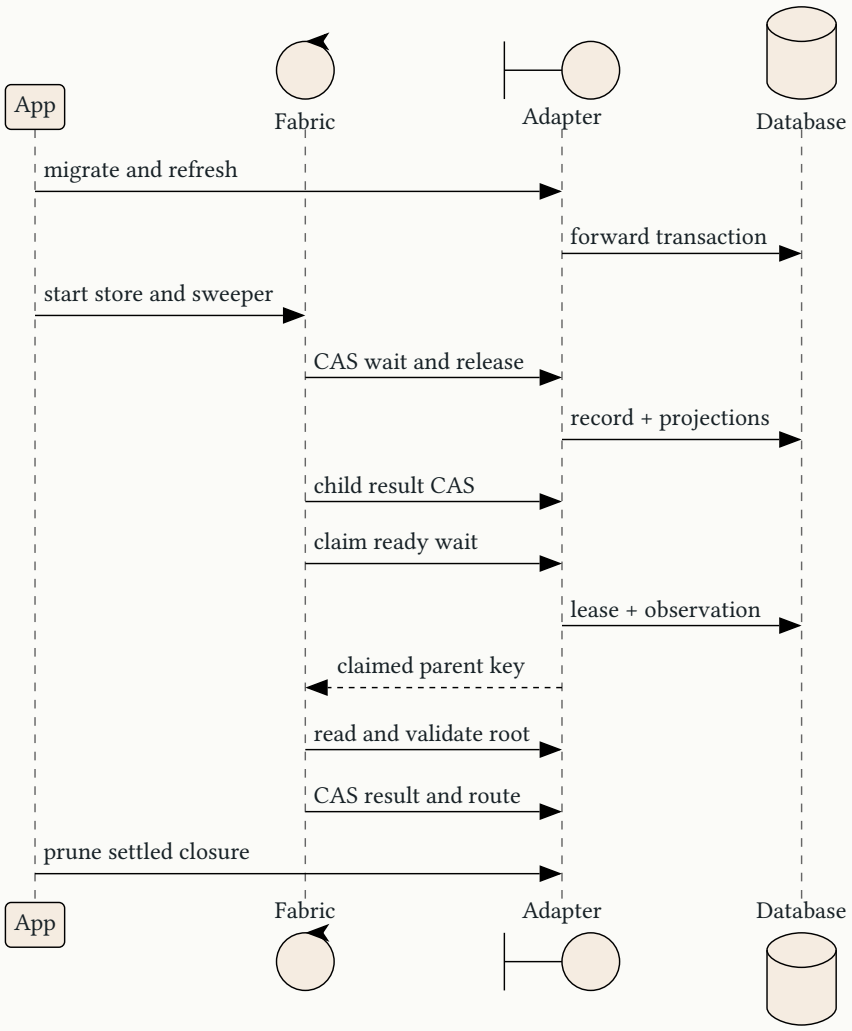
- The disposable cluster disables fsync and synchronous_commit for test speed. Its passing crash/recovery scenarios do not establish power-loss durability; production durability depends on database configuration, replication and the application's operational policy.
- The adapter targets Erlang, Gleam 1.18 or later and PostgreSQL 16 in the retained shell. It constrains pog to 4.1 and pgo to 0.20 minor ranges to compose with applications sharing the pool with Grind; this is dependency compatibility rather than a Grind ownership dependency.
- No adapter LISTEN/NOTIFY listener, global admission queue, fairness guarantee, exactly-once external effect or load/soak certificate is implemented. Cross-node awaits and recovery use parent polling/scans.
- The parent invokes the shared design-layer checker for both layers. Application tests cover compiled consumers and this adapter's disposable database; source-text equality is maintained through review.
- Parent pending capabilities such as progress streaming, whole-run elapsed bounds, family token limits and queue-driven sweeper ownership remain visible in the parent Fabric design layer. None becomes an adapter guarantee merely because records are durable.

ADR-0006 ADR-0007

1.14 End-to-end walkthrough

A retained child wait recovers without rewriting its evidence or resetting its age.

SEQUENCE Claim a changed wait and commit its original successor



The app reconstructs live context; the adapter never stores it. A failed observation leaves an expiry retry path. A claimed key does not authorize a new effect before the parent's confirmed transition.

- An application supplies its named pool and stable node identity api-1. It chooses a 30-second lease and schema public, applies migration 7 and refreshes stale views in positive bounded batches.
- A graph root waits on two admitted children. Its committed discovery projection holds both unfinished child ids; the parent is free and uses no permanent runner while parked.
- The store process is lost after a child commits. On the next scan a changed revision map makes the parent's wait eligible; the ready claim atomically saves that map and acquires the lease without changing parent revision or age.
- The registered root rebuilds compatible graph/agent runtimes and context, validates saved attachments and reads each child's outcome. An uncertain child remains evidence requiring reconciliation; it cannot become a successful join merely from lease expiry.
- The accepted join writes its result, next application state and route through one expected-revision commit. Only confirmation releases the successor. Lost acknowledgements use the retained write token and bytes, without repeating earlier inference.
- Once every family obligation settles and each record is old enough, prune selects the root closure and removes it atomically. Its result counts all removed rows; the application treats a lost commit response as unknown count and ends id reuse for that logical execution.

Glossary

Every term the layer declares, with the context that owns it.

Backend time

owned by root

UTC Unix milliseconds from the database clock that judges lease expiry and automatic wait eligibility. Clock correction can make consecutive samples decrease.

Conditional commit

owned by root

A record update that atomically requires the expected storage revision and the declared lease condition.

Diagnostic snapshot

owned by root

One database observation of classified records, intervention flags, record ages and lease backlog at one sampled time. It authorizes no execution or maintenance.

Lease owner

owned by root

The node, store name and random process identity identifying one Fabric store incarnation. Node identity alone does not identify the process.

Metadata projection

owned by root

A derived interpretation of supported record bytes, tagged with a projection version and the storage revision inspected. It is not another execution record.

PostgreSQL settings

owned by root

The immutable adapter configuration containing a borrowed connection, node identity, lease duration and schema identifier.

Pruning closure

owned by root

The complete stored membership reachable from a Fabric root through validated reciprocal attachments, including its matching family budget record when declared. The parent Fabric layer owns family semantics.

Ready claim

owned by root

A lease acquisition for a free wait whose validated dependency, polling or deadline trigger is eligible. Eligibility supplies a reason to inspect the run, not authority to execute it.

Record age

owned by root

Elapsed backend time since the latest execution-record write. Lease renewal, discovery and metadata refresh do not restart it.

Record bytes

owned by root

The exact encoded Fabric record retained as text, including its write token. Equivalent decoded values do not establish byte identity.

Schema migration

owned by root

One numbered transition of the storage schema, recorded after its statements apply. Its number is independent of record and projection versions.

Storage revision

owned by root

The positive counter identifying one committed version of a stored row. It is independent of record format version and execution incarnation.

Stored lease

owned by root

The optional owner and absolute expiry associated with a stored row. A lease is live only while its expiry is ahead of the backend clock.

Stored row

owned by root

The persistence aggregate for one Fabric record, addressed by its storage key and advanced by an expected revision.

Unknown projection

owned by root

A current-version metadata marker whose source record has no validated interpretation. It grants no pruning or scheduling permission.

Wait observation

owned by root

The last claimed observation key, dependency revision map and backend claim time associated with a discoverable wait.